Homework Assignment: Alibaba Cloud CDN with GitHub Pages

Self-Directed Practical Assignment

Estimated active work: 2–3 hours (excluding account approval, DNS propagation, and service activation)
Submission: One PDF report | Individual work | Suggested score: 100 marks

Purpose: Build a static GitHub Pages website, use it as the origin for Alibaba Cloud CDN, configure caching, verify delivery, update content, purge the cache, and submit evidence that can be independently checked.

1. Important: This Is a Self-Directed Homework Assignment

You must create and manage all required resources yourself. Read the prerequisites before starting.

2. Prerequisites and Cost Controls

- A personal GitHub account.
- An Alibaba Cloud account that can activate CDN. Complete any account or identity requirements shown by the console.
- A domain name that you legally control, plus access to its authoritative DNS settings. Use a subdomain such as cdn.yourdomain.example; do not use a domain you do not own.
- A payment method or available Alibaba Cloud credit if the service requires it. CDN, DNS, domain registration, and certificates may create charges.
- A computer with a modern browser and either curl, PowerShell, or a terminal.
- Permission to publish all uploaded images and PDF files.

Cost safety: Use only a tiny static site, do not run traffic generators, review current pricing before activation, configure billing alerts if available, and disable or delete the CDN domain after grading if you no longer need it. Stop if the console requests a purchase you do not understand.

Regional selection: For this exercise, select Global (Excluding the Chinese Mainland) unless you already have the filing required for another region. Alibaba Cloud documentation states that Chinese Mainland Only and Global require an ICP filing, while Global (Excluding the Chinese Mainland) does not.

3. Architecture and Naming Plan

Choose your own values and record them before configuration:

Item	Your value / example
GitHub username	yourusername
Repository	yourusername.github.io
Origin URL	https://yourusername.github.io
Root domain you control	example.com
Accelerated subdomain	cdn.example.com

Verification token	STUDENTID-CDN-YYYYMMDD

```
User → cdn.example.com → Alibaba Cloud CDN POP → yourusername.github.io
```

Do not: Do not configure cdn.example.com as the GitHub Pages custom domain for this assignment. The subdomain must point to the Alibaba CDN CNAME. GitHub Pages remains the origin at yourusername.github.io.

4. Deliverables

- A public GitHub repository and working GitHub Pages origin.
- A working accelerated subdomain routed through Alibaba Cloud CDN.
- Five cache rules and an explanation of their priorities.
- Command-line and browser evidence comparing origin and CDN behavior.
- A Version 1 → Version 2 update and cache refresh test.
- Monitoring evidence or a documented monitoring-delay fallback.
- One PDF report named StudentID_StudentName_AlibabaCDN_GitHubPages.pdf.

5. Part A — Create the GitHub Pages Origin

Step 1 — Create the repository

1. Sign in to GitHub and create a new repository named exactly YOUR-USERNAME.github.io. Use lowercase where required.
2. Set it to Public when using GitHub Free.
3. Initialize it with a README, then create the repository.

Step 2 — Create the required structure

```
/
├── index.html
├── css/style.css
├── js/app.js
├── images/logo.png
├── api/news.json
└── downloads/guide.pdf
```

Step 3 — Create index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Alibaba Cloud CDN Homework</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="container">
    <img class="logo" src="images/logo.png" alt="Student logo">
    <h1>Alibaba Cloud CDN Homework</h1>
    <p>Hosted on GitHub Pages and delivered through Alibaba Cloud CDN.</p>
    <section>
      <h2>Student Verification</h2>
      <p><strong>Name:</strong> YOUR NAME</p>
      <p><strong>Student ID:</strong> YOUR ID</p>
      <p><strong>Verification token:</strong> YOUR-ID-CDN-YYYYMMDD</p>
```

```
      <p id="version"><strong>Version:</strong> 1</p>
      <p><strong>Last updated:</strong> YYYY-MM-DD HH:MM TZ</p>
    </section>
    <section>
      <h2>JSON Test</h2>
      <p id="news">Loading news.json…</p>
    </section>
    <p><a href="downloads/guide.pdf">Download the guide</a></p>
  </main>
  <script src="js/app.js"></script>
</body>
</html>
```

Step 4 — Create css/style.css

```
body { font-family: Arial, sans-serif; margin: 0; background: #f4f6f8; color: #222; }
.container { width: min(900px, 86%); margin: 40px auto; padding: 28px; background: white; }
.logo { max-width: 150px; height: auto; }
h1 { color: #ff6a00; }
section { margin-top: 22px; padding: 14px; border-left: 4px solid #ff6a00; background: #fafafa;
}
```

Step 5 — Create js/app.js and api/news.json

```
fetch("api/news.json", { cache: "no-store" })
  .then(response => {
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return response.json();
  })
  .then(data => document.getElementById("news").textContent = data.message)
  .catch(error => document.getElementById("news").textContent = `Load failed:
${error.message}`);
```

```
{
  "message": "This message was loaded from news.json Version 1.",
  "version": 1
}
```

Step 6 — Add your own media and commit

4. Upload an image you created or are permitted to use as images/logo.png.
5. Create a short, non-confidential PDF named downloads/guide.pdf.
6. Commit all files with the message: Create Version 1 GitHub Pages origin.

Step 7 — Publish GitHub Pages

7. Open Repository Settings → Pages.
8. Under Build and deployment, choose Deploy from a branch.
9. Choose main and /(root), then Save.
10. Wait for deployment and open https://YOUR-USERNAME.github.io.
11. Verify your name, ID, token, Version 1, image, JSON message, and PDF link.

```
curl -I https://YOUR-USERNAME.github.io/
curl -I https://YOUR-USERNAME.github.io/images/logo.png
curl -I https://YOUR-USERNAME.github.io/api/news.json
```

Evidence A: Repository file list and commit ID; Pages settings; origin page with complete URL and token; curl output for the origin. Explain what each item proves.

6. Part B — Configure Alibaba Cloud CDN

Step 8 — Activate CDN and verify domain ownership

12. Sign in to Alibaba Cloud and activate CDN only after reviewing current pricing.
13. Open CDN → Domain Names → Add Domain Name.
14. If prompted to verify ownership, use the DNS TXT method: copy the exact host and value shown by Alibaba Cloud into your DNS provider, wait for propagation, and click Verify.
15. Keep evidence of the TXT record and successful verification. You may remove the TXT record later only if the console permits it.

Step 9 — Add the accelerated domain

16. Use a subdomain you control, for example cdn.example.com.
17. Choose Global (Excluding the Chinese Mainland) unless you already meet the filing requirement for another region.
18. Choose the business type for images and small files.
19. Proceed to origin configuration.

Step 10 — Configure GitHub Pages as the origin

```
Origin type: Domain name
Origin address: YOUR-USERNAME.github.io
Origin protocol: HTTPS
Origin port: 443
Back-to-origin Host / Origin Host: YOUR-USERNAME.github.io
```

Critical: Enter only the hostname in a hostname field—do not include https:// or a path. The origin Host must be YOUR-USERNAME.github.io because GitHub Pages uses the requested hostname to select the correct site.

Validation: Before changing DNS, use any Alibaba Cloud origin-testing feature offered by the console. A GitHub 404 usually indicates an incorrect origin name, repository name, publishing status, path, or Host header.

Step 11 — Record the Alibaba CDN CNAME

After the domain is created, copy the generated CNAME. It commonly resembles cdn.example.com.w.kunlunsl.com, but use the exact value in your console.

Evidence B: Screenshots showing your accelerated domain, region, business type, GitHub origin, HTTPS/443, origin Host, generated CNAME, and successful ownership verification. Hide account identifiers and all secrets.

7. Part C — Route DNS and Configure HTTPS

Step 12 — Create the CNAME record

20. Open the authoritative DNS manager for your root domain.
21. Create a CNAME for the selected host, such as cdn.
22. Set its value to the exact Alibaba Cloud CDN CNAME.
23. Remove conflicting A, AAAA, or CNAME records for that same hostname, after confirming they are not needed.
24. Save the record and wait for propagation.

```
# Windows PowerShell
Resolve-DnsName cdn.example.com -Type CNAME

# Cross-platform alternatives
```

```
nslookup -type=CNAME cdn.example.com
dig cdn.example.com CNAME
```

Step 13 — Configure HTTPS for the accelerated domain

GitHub Pages HTTPS protects the origin connection, but it does not automatically provide a certificate for your Alibaba CDN hostname. Configure HTTPS in Alibaba Cloud CDN for cdn.example.com using a certificate you are authorized to use, following the options offered by your account. Never upload or expose a private key in your report.

25. Open the accelerated domain → HTTPS settings.
26. Select or upload a valid certificate matching your CDN hostname.
27. Enable HTTPS. Enable HTTP-to-HTTPS redirection only after HTTPS works.
28. Open https://cdn.example.com and confirm there is no certificate warning.

Fallback: If certificate issuance is still pending, complete initial functional tests over HTTP and clearly document the limitation. HTTPS evidence is required unless the grading policy explicitly allows the fallback.

Evidence C: DNS record screenshot; DNS command output; CDN status; HTTPS page with complete URL; certificate information showing the hostname and validity dates but no private key.

8. Part D — Configure Cache Expiration

Priority	Match	TTL	Reason
Highest	/api/news.json	1 minute or no-cache	Frequently updated data
High	/index.html	10 minutes	Page updates should appear soon
Medium	jpg, jpeg, png	7 days	Images change infrequently
Medium	css, js	1 day	Static but updated more often than images
Medium	pdf	3 days	Downloads usually change infrequently

29. Open the accelerated domain → Cache Settings or Cache Expiration.
30. Create all five rules using the exact paths/extensions above.
31. Place specific path rules above broad extension or directory rules.
32. Verify that all rules are enabled and save screenshots showing match, TTL, status, and priority.

Evidence D: Readable screenshots of all five cache rules plus one sentence justifying each TTL. Explain what stale-data problem would occur if news.json were cached for seven days.

9. Part E — Verify Origin, CDN, and Cache Behavior

Step 14 — Compare content

```
curl -I https://YOUR-USERNAME.github.io/images/logo.png
curl -I https://cdn.example.com/images/logo.png
```

Open both sites and verify that the student token and Version 1 match. The CDN screenshot must show the CDN hostname, not github.io.

Step 15 — Repeat requests

```
curl -sS -D - -o /dev/null https://cdn.example.com/images/logo.png
curl -sS -D - -o /dev/null https://cdn.example.com/images/logo.png
```

```
curl -sS -D - -o /dev/null https://cdn.example.com/images/logo.png
curl -sS -D - -o /dev/null https://cdn.example.com/api/news.json
```

Inspect HTTP status, Age, Cache-Control, Expires, Via, and any Alibaba Cloud cache-status headers. Header availability and names can vary; do not claim a hit unless the evidence supports it. If the headers are inconclusive, state that and combine them with monitoring evidence.

Step 16 — Browser Developer Tools

33. Open the CDN URL and Developer Tools → Network.
34. Enable Disable cache so the browser's local cache does not hide CDN behavior.
35. Clear the list, reload, and inspect logo.png.
36. Capture request URL, status, timing, response headers, and size.
37. Reload and repeat for comparison.

Evidence E: Origin and CDN page screenshots; complete commands and outputs for repeated image and JSON requests; two Network screenshots; analysis distinguishing browser cache, CDN edge cache, and GitHub Pages origin.

10. Part F — Update GitHub Pages and Refresh CDN Content

Step 17 — Create Version 2

38. Capture Version 1 through the CDN with the full URL visible.
39. Edit index.html: change Version 1 to Version 2 and update the timestamp.
40. Edit news.json: change its message and version to 2.
41. Commit with: Update to Version 2 for CDN refresh test.
42. Wait until https://YOUR-USERNAME.github.io displays Version 2 and capture the commit ID and origin page.

Step 18 — Observe and refresh the CDN

43. Open the CDN URL before purging and record the displayed version. It may already be Version 2 if the TTL expired or revalidation occurred; report the actual result.
44. Open Alibaba Cloud CDN → Refresh/Purge URL.
45. Submit the complete CDN URLs for /index.html and /api/news.json.
46. Wait for completion, access the CDN URL again, and verify Version 2.

Evidence F: Version 1 CDN page; Version 2 GitHub commit and origin page; purge request showing URLs and status; Version 2 CDN page. Include date/time and explain TTL expiration, purge, and preload.

11. Part G — Monitoring and Troubleshooting

Step 19 — Monitoring

47. Generate only a small number of manual requests to each resource.
48. Open Resource Monitoring, select your domain and a time range covering the test.
49. Capture available requests, traffic, back-to-origin traffic, HTTP status codes, and cache-hit metrics.
50. If data is delayed, capture the selected domain/time range and state that curl and browser evidence were used as the immediate fallback.

```
Request hit ratio = (total requests − origin requests) ÷ total requests × 100
Example: (1000 − 250) ÷ 1000 × 100 = 75%
```

Step 20 — Troubleshoot one genuine issue

```
Problem observed:
Expected result:
Actual result:
Possible cause:
Tool or command used:
Evidence before correction:
Corrective action:
Evidence after correction:
Technical explanation:
```

- GitHub 404: check repository name, Pages deployment, path case, origin hostname, and origin Host.
- DNS failure: check CNAME spelling, conflicting records, authoritative nameservers, and propagation.
- Certificate warning: verify certificate hostname, validity, chain, and CDN HTTPS status.
- Old content: compare origin and CDN, check TTL/rule priority, purge the exact URL, and bypass browser cache.
- JSON failure: inspect the Network console, file path, JSON syntax, case sensitivity, and HTTP status.

Evidence G: Monitoring screenshot or documented delay; hit-ratio calculation; one real troubleshooting record with before-and-after evidence.

12. Final Report Instructions

Submit one PDF named:

```
StudentID_StudentName_AlibabaCDN_GitHubPages.pdf
```

- Cover page and resource inventory.
- Architecture diagram and explanation.
- GitHub repository and Pages origin evidence.
- Alibaba Cloud domain, origin, Host, and ownership evidence.
- DNS and HTTPS evidence.
- Cache-rule evidence and justification.
- Origin/CDN comparison and cache tests.
- Version 2 commit and refresh/purge test.
- Monitoring and hit-ratio calculation.
- Troubleshooting record.
- Answers to the final questions.
- Cleanup statement and references.

13. Evidence Acceptance Rules

- Every screenshot must show enough context to identify the service and resource.
- Browser screenshots must show the complete URL. Terminal evidence must show the command and output.
- Your GitHub username, student ID, verification token, accelerated hostname, and commit ID must be visible where relevant.
- Caption every figure and explain in one or two sentences what it proves.
- Do not expose passwords, access keys, account numbers, billing details, certificate private keys, or session cookies.
- Do not edit evidence except to redact sensitive information. Clearly label redactions.
- A configuration screenshot alone does not prove operation; include independent DNS, HTTP, and content evidence.

- Retain the repository and CDN configuration until grading is complete, unless cost or security requires earlier removal.

14. Final Questions

1. What roles do GitHub Pages and Alibaba Cloud CDN perform?

2. Why must the accelerated hostname point to Alibaba Cloud's CNAME rather than directly to username.github.io?

3. Why is the back-to-origin Host set to username.github.io?

4. What is the difference between cache hit, cache miss, browser cache, and CDN edge cache?

5. Why do images, HTML, and JSON use different TTL values?

6. Why can GitHub Pages show Version 2 while the CDN still shows Version 1?

7. What is the difference between TTL expiration, purge/refresh, and preload?

8. What evidence proves the request used Alibaba Cloud CDN?

9. What is the difference between request hit ratio and byte hit ratio?

10. If CDN requests increase while back-to-origin traffic stays low, what does that suggest?

11. What security and cost risks exist in this homework, and how did you control them?

15. Student Checklist

☐ Own domain and DNS control

☐ GitHub Pages Version 1

☐ Unique token

☐ CDN origin and Host configured

☐ Ownership verified

☐ CNAME resolves

☐ HTTPS works

☐ Five cache rules

☐ Repeated curl tests

☐ Developer Tools evidence

☐ Version 2 commit

☐ Purge evidence

☐ Monitoring or delay note

☐ Troubleshooting before/after

☐ Questions answered

☐ Secrets redacted

☐ PDF filename correct

☐ Cleanup completed or scheduled

16. References

- GitHub Docs — Creating a GitHub Pages site: https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site
- GitHub Docs — Configuring a publishing source: https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site
- Alibaba Cloud CDN — Add a domain name: https://www.alibabacloud.com/help/en/cdn/add-a-domain-name
- Alibaba Cloud CDN — Verify domain ownership: https://www.alibabacloud.com/help/en/cdn/verify-domain-name-ownership
- Alibaba Cloud CDN — Configure a CNAME: https://www.alibabacloud.com/help/en/cdn/add-a-cname-record-for-a-domain-name
- Alibaba Cloud CDN — Resource monitoring: https://www.alibabacloud.com/help/en/cdn/user-guide/resource-monitoring